Week 1. AI 협업이 바꾸는 개발 프로세스

> 원본: docs/ch1.md

학습 목표

- AI와 협업하는 개발에서 사람이 책임져야 하는 판단과 검증의 범위를 설명한다
- 에이전틱 코딩 도구의 유형(IDE 기반, 터미널 기반)과 선택 기준을 이해한다
- 바이브 코딩의 개념과 실무 한계를 파악한다
- 환각, 과신, 컨텍스트 누락 같은 실패 패턴을 식별하고 대응 원칙을 정리한다

선수 지식

- 기본적인 파이썬 실행 경험이 있으면 충분함
  - 가상환경 설정, 패키지 설치 수준
- 본 장의 목적
  - 특정 프레임워크를 깊게 다루지 않음
  - 이후 장에서 반복될 "검증 가능한 개발 흐름"을 먼저 만드는 데 집중

1.1 개발 패러다임의 변화: 타이핑에서 설계·검증으로

- AI 코딩 도구를 처음 접할 때 눈에 띄는 변화
  - "코드를 대신 써준다"는 점이 가장 두드러짐
- 그러나 실무에서의 더 큰 변화
  - 타이핑 속도 자체가 빨라지는 것이 아님
  - 설계 대안을 탐색하고 검증을 반복하는 비용이 크게 낮아지는 것
  - 같은 시간 안에 가능해지는 것들:
    - 더 많은 가설을 세울 수 있음
    - 더 많은 구현 후보를 만들 수 있음
    - 더 많은 검증을 수행할 수 있음
  - 결과: 실패를 더 빨리 발견할 가능성이 커짐
- AI 시대에서 개발자의 역할 변화
  - 역할이 사라지지 않음 — 오히려 더 중요해짐
  - 더 중요해지는 책임:
    - 요구사항을 명확히 하는 일
    - 위험을 판단하는 일
    - 검증 기준을 설계하는 일
  - AI가 할 수 있는 것
    - "좋아 보이는 코드"를 제안하는 것
  - ⚠ AI가 할 수 없는 것
    - 제안한 코드가 현재의 제약을 만족하는지 확인하는 것

        - 제약의 종류: 환경, 버전, 보안 정책, 운영 조건
        - 이 확인은 반드시 사람이 해야 함

- 에이전틱(agentic) 단계로의 진입 (2024년~)
    - 이전 단계: 단순한 코드 자동 완성
    - 새로운 단계 (에이전틱): 다음을 스스로 수행
        - 여러 파일을 동시에 수정
        - 터미널 명령을 실행
        - 오류를 스스로 수정
    - 이 변화가 개발자에게 의미하는 바
        - 생성 능력이 강해질수록
        - 무엇을 생성할지 판단하는 능력이 차별화 요인이 됨
        - 생성된 결과를 검증하는 능력이 차별화 요인이 됨

- 이 강의의 관점
    - 위의 관점을 출발점으로 삼음
    - MCP와 에이전트 프레임워크를 다루면서도
    - 언제나 "검증 가능한 산출물"을 남기는 습관을 기본값으로 유지

1.2 에이전틱 코딩 도구 전경

- 2026년 3월 기준, 주요 AI 코딩 도구는 "채팅 보조"를 넘어 비동기 작업 위임, IDE 통합, CLI 에이전트, MCP 연동으로 빠르게 확장 중
- ⚠ 이 영역은 제품 출시 주기가 매우 빠르므로, 요금제·무료 한도·실험 기능 여부는 항상 공식 문서를 다시 확인해야 한다
- 에이전트 모드란? (개발자가 목표를 제시하면 도구가 자율적으로 작업하는 방식)
    - 도구가 하는 일:
        - 코드베이스를 분석
        - 여러 파일을 수정
        - 터미널 명령을 실행
        - 오류를 자체 수정
    - 기존의 "한 줄 자동 완성"과는 근본적으로 다른 패러다임

1.2.1 IDE 기반 도구

- IDE 기반 도구의 특징
    - 에디터 안에서 통합적으로 수행:
        - 코드 편집
        - 파일 탐색
        - 터미널 실행

- **GitHub Copilot 에이전트 모드**
    - 2025년 2월 VS Code 기반 에이전트 모드를 공개했고, 2025년 9월 GitHub Copilot coding agent를 GA(일반 공개)했다
    - 자율 루프 기능:

        - 코드베이스를 분석
        - 여러 파일에 걸친 수정을 제안
        - 터미널에서 명령을 실행
        - 테스트가 통과할 때까지 반복
    - 클라우드 기반 코딩 에이전트(Coding Agent):
        - GitHub Issues를 할당받아 자율적으로 드래프트 PR을 생성
        - GitHub Actions 기반 개발 환경에서 비동기로 실행
    - 2026년 3월 기준 메모
        - GitHub Docs 기준으로 Copilot Pro, Pro+, Business, Enterprise 플랜에서 사용 가능
        - 세부 사용량은 GitHub Actions 분과 Copilot premium requests에 의해 과금/차감될 수 있음

- **Cursor**
    - 특징: VS Code를 포크(fork, 소스코드를 복사하여 별도로 개발)한 전용 IDE
    - 출시: 2023년 공개 이후 빠르게 발전한 전용 IDE
    - 핵심 기능:
        - Composer 모드: 여러 파일을 동시에 편집
        - 에이전트형 편집 및 코드베이스 탐색 기능
    - ⚠ 세부 요금제와 병렬 작업 수는 자주 바뀌므로 수업 시점에 공식 제품 문서를 다시 확인할 것

- **Windsurf**
    - 출시: Codeium이 2024년 11월에 출시한 IDE
    - 핵심 기능: Cascade
        - 이중 구조: 장기 계획 에이전트 + 단기 실행 모델이 분리됨
        - 실시간으로 코드베이스 상태를 추적하면서 다단계 작업을 수행
    - ⚠ 세부 크레딧 정책과 플랜 구성은 자주 바뀌므로 공식 문서 확인이 필요

1.2.2 터미널 기반 도구

- 터미널 기반 도구의 특징
    - 명령줄(CLI)에서 동작
    - 기존 워크플로우와 자연스럽게 통합:
        - git
        - 셸 스크립트
        - 기타 CLI 도구들

- **Claude Code**
    - Anthropic의 터미널 기반 에이전틱 코딩 도구
    - 동작 방식:
        - 터미널에서 자연어로 지시
        - 코드베이스 전체를 이해
        - 수행 가능한 작업: 파일 읽기/쓰기, git 작업, 셸 명령 실행
    - 2026년 3월 기준 메모
        - 공식 문서는 Claude Code를 "터미널에 사는 agentic coding tool"로 설명
        - MCP를 통해 외부 도구와 데이터 소스를 연결할 수 있고, VS Code·JetBrains 계열 IDE와도 연동 가능

- **Gemini CLI**
  - 출시: Google이 2025년 6월 공개한 오픈소스 CLI
  - 라이선스: Apache 2.0 오픈소스
  - 장점:
    - 개인 Google 계정으로 시작 가능
    - MCP 서버 연동 기본 지원
    - 100만 토큰 컨텍스트 윈도우 활용 가능
  - 2026년 3월 기준 메모
    - Google 발표 기준 프리뷰 단계에서 분당 60회, 일 1,000회 무료 요청 한도를 제공
- **OpenAI Codex CLI**
  - 2025년 4월 공개된 오픈소스 CLI
  - 개발 언어: Rust 기반
  - 특징:
    - 로컬에서 파일 수정, 명령 실행, 테스트 수행이 가능한 경량 코딩 에이전트
    - 샌드박스(sandbox, 격리된 안전 실행 환경)와 승인 워크플로우를 제공
  - 2026년 3월 기준 메모
    - OpenAI는 Codex를 CLI뿐 아니라 IDE 확장, 웹, 데스크톱 앱으로 확장 중
    - ChatGPT 계정 로그인 또는 API 키 방식으로 사용할 수 있음

1.2.3 도구 선택 기준

- 선택 기준: 프로젝트 성격과 팀 워크플로우에 따라 달라짐
- IDE 기반 도구가 적합한 경우
  - 강점: 시각적 피드백, 에디터 통합
  - → 예시: 프론트엔드 개발, 빠른 프로토타이핑
- 터미널 기반 도구가 적합한 경우
  - 강점: 기존 CLI 워크플로우와 자연스럽게 결합
  - → 예시: 백엔드 개발, 대규모 리팩토링

표 1.1 에이전틱 코딩 도구 비교 (2026년 3월 점검판)

도구	유형	기준 시점	접근 방식	핵심 특징
GitHub Copilot coding agent	IDE + GitHub 클라우드	GA: 2025.09	유료 Copilot 플랜	GitHub Actions 기반 비동기 작업 위임
Cursor	IDE (VS Code 포크)	2026.03 기준 사용 중	벤더 플랜 확인 필요	에이전트형 편집, 코드베이스 탐색
Windsurf	IDE (VS Code 포크)	2026.03 기준 사용 중	벤더 플랜 확인 필요	Cascade 기반 다단계 작업
Claude Code	터미널 (CLI)	2026.03 공식 문서 기준	Claude 계정/조직 설정	터미널 네이티브 + MCP + IDE 연동

도구	유형	기준 시점	접근 방식	핵심 특징
Gemini CLI	터미널 (오픈소스)	공개: 2025.06	개인 Google 계정 또는 API 키	Apache 2.0 + MCP + 1M 컨텍스트
OpenAI Codex CLI	터미널 (오픈소스)	공개: 2025.04, 제품 확장: 2026.02~03	ChatGPT 로그인 또는 API 키	Rust 기반 + 샌드박스 + IDE/앱 연동

- 어떤 도구를 선택하든 핵심은 동일함
  - 에이전트가 생성한 코드를 사람이 검증하는 구조를 유지하는 것
  - ⚠ 도구의 자율성이 높아질수록 검증 습관의 중요성도 비례하여 커짐

1.3 바이브 코딩의 가능성과 한계

1.3.1 바이브 코딩이란

- 용어의 유래
  - 최초 사용자: 안드레이 카르파시(Andrej Karpathy, OpenAI 공동 창업자)
  - 발표 시점: 2025년 2월, X(구 트위터)에 올린 글
  - 원문 묘사: "코드가 존재한다는 사실조차 잊고, 분위기(vibe)에 맡긴다"
- 바이브 코딩의 방식
  - AI에게 자연어로 원하는 것을 말함
  - 생성된 코드를 읽지 않고 모두 수락
  - 오류가 나면 오류 메시지를 그대로 붙여넣어 해결
- 확산 속도
  - 수주 만에 주요 매체에 소개됨: 뉴욕 타임스, 아스 테크니카, 가디언
  - 위키피디아 문서가 생성될 만큼 널리 퍼짐
- 바이브 코딩의 가능성
  - 적합한 상황:
    - 프로토타입을 빠르게 만들어야 할 때
    - 일회성 스크립트가 필요할 때
  - 진입 장벽을 극적으로 낮춤
  - 프로그래밍 경험이 적은 사람도 동작하는 소프트웨어를 만들 수 있음

1.3.2 실무에서의 한계

- ⚠ 바이브 코딩을 프로덕션(실제 서비스 운영) 코드에 무비판적으로 적용하면 위험이 커짐
- 다만 아래 사례 중 일부는 **2025년 시점의 연구와 관찰**에 근거하므로, 현재의 모든 AI 도구에 그대로 일반화해서는 안 됨
- 핵심 경고는 "AI를 쓰지 말라"가 아니라, **코드를 읽지 않고 배포하지 말라**는 데 있음
- 첫 번째 문제: 보안 취약점

    - 2025년 보안 보고서들은 AI 생성 코드가 알려진 웹 취약점을 포함하는 사례를 반복적으로 보고함
    - 특히 SQL 인젝션, XSS 같은 기본적인 입력 검증 실패가 문제로 자주 언급됨
    - 코드를 읽지 않고 수락하면:
        - SQL 인젝션(악의적 SQL 명령을 삽입하는 공격) 취약점이 프로덕션에 배포될 수 있음
        - XSS(크로스 사이트 스크립팅, 웹 페이지에 악성 스크립트를 삽입하는 공격) 취약점이 포함될 수 있음
        - 인증·권한 처리 로직이 불완전한 상태로 들어갈 수 있음
    - 주의할 점:
        - "AI가 만든 코드는 위험하다"가 아니라, **검토하지 않은 코드는 위험하다**는 것이 더 정확한 표현
        - 모델 성능이 개선되더라도 보안 검토 책임은 여전히 사람에게 있음

- 두 번째 문제: 유지보수 불가
    - 바이브 코딩으로 만든 코드베이스는 구조 일관성이 쉽게 무너질 수 있음
    - 결과:
        - 개발자 자신도 코드의 동작 원리를 이해하지 못하는 상태가 됨
        - 버그 수정이나 기능 추가 때 부분 수정 대신 재생성을 반복하게 될 수 있음
    - 전문가 의견 — 사이먼 윌리슨(Simon Willison):
        - "우리가 소프트웨어 엔지니어로서 하는 일의 대부분은 기존 시스템을 진화시키는 것"
        - "이때 코드의 품질과 이해 가능성이 결정적"
    - 현재 시점의 해석:
        - 최신 도구는 예전보다 긴 맥락과 다중 파일 편집을 더 잘 다루지만, **이해하지 못한 코드는 여전히 유지보수 비용을 키움**

- 세 번째 문제: 생산성 착각
    - 근거: METR의 2025년 연구
        - 대상: 숙련된 오픈소스 개발자 16명
        - 작업: 실제 작업 246건 수행
        - 결과: AI 도구를 사용했을 때 오히려 19% 느려짐
    - 하지만 이 결과는 **초기 2025년 도구 성능 + 숙련 개발자 + 익숙한 오픈소스 저장소 작업**이라는 조건에서 나온 것
    - 따라서 이 연구를 근거로 "AI를 쓰면 항상 느려진다"라고 말할 수는 없음
    - ⚠ 흥미로운 점 — 체감과 실제의 괴리:
        - 개발자들이 스스로 예측한 것: 24% 빨라질 것이라고 예측
        - 실험 후 개발자들이 느낀 것: 20% 빨라졌다고 느낌
        - 실제 측정 결과: 19% 느려짐
        - 체감 속도와 실제 속도 사이에 상당한 괴리가 존재
    - 현재 시점의 해석:
        - 최신 모델과 에이전트 도구는 당시보다 분명히 발전했음
        - 그럼에도 생산성은 작업 종류, 개발자의 숙련도, 검토 비용에 따라 크게 달라짐
        - 따라서 "빨라졌는가"는 느낌이 아니라 실제 측정으로 확인해야 함

1.3.3 AI 보조 프로그래밍과의 구분

- 바이브 코딩 vs AI 보조 프로그래밍(AI-assisted programming) — 다른 개념

- AI 보조 프로그래밍의 특징

    - 개발자가 생성된 코드를 이해함
    - 개발자가 코드를 검토함
    - 필요하면 수정함
    - 주도권이 사람에게 있음

- 바이브 코딩의 특징
    - 코드 이해를 포기
    - 결과만 확인하는 방식

- 이 책의 입장
    - 모든 실습은 AI 보조 프로그래밍 관점을 따름
    - 에이전트가 코드를 생성하더라도:
        - 사람이 설계를 판단하는 구조를 유지
        - 사람이 결과를 검증하는 구조를 유지

1.4 AI 협업의 역할 분담: 무엇을 맡기고 무엇을 검증하는가

- AI 협업을 잘하는 방법의 순서
    - 먼저: "무엇을 맡기고 무엇을 남길지"를 정하는 일
    - 그 다음: "잘 시키기"

- AI가 잘하는 영역 (맡길 수 있는 작업)
    - 초안 생성
    - 대안 탐색
    - 리팩토링 후보 제안
    - 공통점: 탐색적 작업

- 사람이 책임져야 하는 영역 (남겨야 하는 작업)
    - 요구사항 확정
    - 위험 판단
    - 검증 설계
    - 운영 책임
    - 공통점: 결과의 책임이 따르는 작업

- 팀의 규칙으로 만들기 위한 습관
    - 작업 산출물을 '문장'이 아니라 '파일'로 남기기
    - 파일로 남겨야 할 내용:
        - 어떤 코드를 선택했는지
        - 어떤 입력에서 확인했는지
        - 어떤 실패를 고려했는지
    - 파일로 남겼을 때의 이점: 다음 단계가 쉬워짐
        - 리뷰
        - 수정
        - 재현

표 1.2 사람과 AI의 역할 분담(권장)

구분	사람이 담당	AI가 보조(권장)
목표/범위	문제 정의, 완료 조건 확정	요구사항 질문 목록 제안
설계	제약/리스크 판단, 의사결정	대안 아키텍처 초안 제안
구현	핵심 로직 검증, 통합/운영 고려	코드 초안 생성, 리팩토링 후보
검증	테스트/재현 절차 정의, 결과 해석	테스트 코드 초안, 에러 원인 후보
안전	비밀정보/권한/파괴적 동작 통제	체크리스트 제안, 누락 탐지

1.5 실패 패턴과 대응: 환각/과신/컨텍스트/보안

- AI 협업의 실패 원인
  - "답이 틀렸다"가 아님
  - 대체로 "확인 없이 믿었다"에서 시작
- 실패 패턴 1: 환각(hallucination, AI가 없는 것을 있다고 말하는 현상)
  - 형태: 존재하지 않는 API, 옵션, 참고자료를 사실처럼 말함
  - → 예시: 존재하지 않는 라이브러리 함수를 자신 있게 제안
  - → 예시: 가짜 DOI가 포함된 논문을 인용
- 실패 패턴 2: 과신(overconfidence, AI가 틀렸는데도 확신하는 현상)
  - 형태: "된다"는 주장과 실제 실행의 괴리가 커짐
  - → 예시: AI가 "이 코드는 모든 엣지 케이스를 처리합니다"라고 주장
  - ⚠ 실제 실행 없이는 확인할 수 없음
- 실패 패턴 3: 컨텍스트 누락
  - 원인: 파일 구조, 버전, 운영 환경 차이를 반영하지 못함
  - → 예시: 프로젝트에 이미 존재하는 유틸리티 함수를 무시하고 새로 작성
  - → 예시: 프로덕션 환경의 제약을 고려하지 않는 코드를 제안
- 실패 패턴 4: 보안 문제
  - 형태: 비밀정보 노출이나 파괴적 동작을 무심코 실행
  - → 예시: API 키를 코드에 하드코딩(직접 삽입)
  - → 예시: `rm -rf` 같은 위험한 명령을 검증 없이 제안
- 이를 막기 위한 최소 규칙 (4가지)
  - 첫째: 목표와 산출물을 합의한다
  - 둘째: 입력과 제약을 확인한다
  - 셋째: 실행 가능한 재현 절차를 남긴다
  - 넷째: 안전과 보안을 기본값으로 둔다
- 다음 절 미니 실습의 목적

    - 위 규칙을 "체크리스트 파일"로 만들어 프로젝트 산출물로 남기는 것

1.6 미니 실습: 검증 체크리스트 파일 자동 생성

- 이 절의 목표
    - 복잡한 작업 자동화를 만드는 것이 목적이 아님
    - 검증 항목의 틀을 파일로 자동 생성해, 이후 실습에서 반복 사용할 수 있게 만드는 것
    - 자동화를 만들 때 반드시 따라야 하는 검증 습관을 파일 산출물로 남기는 것
- 실습 결과물
    - 실습 스크립트를 실행하면 체크리스트 문서 템플릿이 생성됨
    - 저장 위치: `practice/chapter1/data/output/ch01_checklist.md`
- 이 실습에서 자동화되는 것
    - 사람이 빈 문서를 직접 작성하지 않아도 됨
    - 작업 범위, 입력·제약, 검증, 안전 항목이 포함된 기본 체크리스트가 자동으로 파일로 생성됨
    - 이후 실습에서는 이 파일을 복사하거나 수정하여 검증 기록의 출발점으로 사용 가능
- 이 체크리스트를 사용하는 방식
    - 작업 전: 이번 작업의 범위, 입력, 제약, 검증 방법, 안전 항목을 먼저 적음
    - 작업 후: 실제 결과물을 같은 체크리스트 기준으로 다시 확인함
    - 즉, 이 문서는 단순 메모가 아니라 **작업 전 계획표이자 작업 후 검수표** 역할을 함
- 전체 코드 참고: `practice/chapter1/code/1-4-checklist-generator.py`
- 본문에서 확인할 핵심 코드 (출력 폴더 생성 및 파일 쓰기)

```
output_dir = chapter_dir / "data" / "output"
output_dir.mkdir(parents=True, exist_ok=True)
output_path = output_dir / "ch01_checklist.md"
output_path.write_text(CHECKLIST_TEXT, encoding="utf-8")
```

- ⚠ 실습 후 주의사항
    - 생성된 체크리스트를 만드는 것만으로 끝내면 안 됨
    - 이후 장의 실습에서도 아래 항목을 실제로 채워 넣는 방식으로 습관화해야 함:
        - 작업 범위
        - 입력·제약
        - 검증
        - 안전

핵심 정리

- AI 협업 개발의 핵심은 "빠른 생성"이 아니라 "빠른 검증"에 있다
- 에이전틱 코딩 도구의 두 유형:

    - IDE 기반: Copilot, Cursor, Windsurf
    - 터미널 기반: Claude Code, Gemini CLI, Codex CLI
    - 프로젝트 성격에 따라 선택
- 바이브 코딩의 위치:
    - 프로토타입에는 유효함
    - ⚠ 프로덕션 코드에는 한계가 명확함: 보안, 유지보수, 생산성 모든 측면에서
- 사람의 책임 (요구사항, 위험 판단, 검증 설계):
    - 줄어들지 않음
    - 오히려 더 중요해짐
- 산출물을 남기는 방식:
    - 텍스트 주장 → 재현과 검토가 어려움
    - 파일로 남김 → 재현과 검토가 가능해짐

다음 장 예고

- 2장의 주제: 로컬 개발 환경과 재현 가능한 실행
- 다룰 내용:
    - Python 가상환경
    - 의존성 고정
    - 크로스 플랫폼 경로 처리
- 목적: 이후 모든 실습의 기반을 마련
- 이 장에서 만든 체크리스트는 이후 실습의 검증 습관으로 계속 사용됨

참고문헌

Karpathy, A. (2025). Vibe Coding. X (formerly Twitter). https://x.com/karpathy/status/1886192184808149383

Willison, S. (2025). Not all AI-assisted programming is vibe coding. *Simon Willison's Weblog*. https://simonwillison.net/2025/Mar/19/vibe-coding/

Christiano, P. et al. (2025). Measuring the Impact of Early 2025 AI Models on Experienced Open-Source Developer Productivity. *METR*. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

GitHub. (2025). Introducing Copilot Agent Mode. *VS Code Blog*. https://code.visualstudio.com/blogs/2025/02/24/introducing-copilot-agent-mode

GitHub. (2025). About GitHub Copilot coding agent. *GitHub Docs*. https://docs.github.com/en/copilot/concepts/coding-agent/coding-agent

GitHub. (2025). Copilot coding agent is now generally available. *GitHub Changelog*. https://github.blog/changelog/2025-09-25-copilot-coding-agent-is-now-generally-available/

Anthropic. (2025). Claude Code overview. *Anthropic Docs*. https://docs.anthropic.com/en/docs/claude-code/overview

Anthropic. (2025). Add Claude Code to your IDE. *Anthropic Docs*. https://docs.anthropic.com/en/docs/claude-code/ide-integrations

Google. (2025). Introducing Gemini CLI. *Google Blog*. https://blog.google/innovation-and-ai/technology/developers-tools/introducing-gemini-cli-open-source-ai-agent/

OpenAI. (2025). Introducing Codex. *OpenAI*. https://openai.com/index/introducing-codex/

OpenAI. (2026). Introducing the Codex app. *OpenAI*. https://openai.com/index/introducing-the-codex-app/

OpenAI. (2026). OpenAI Codex CLI – Getting Started. *OpenAI Help Center*. https://help.openai.com/en/articles/11096431-openai-codex-cli-getting-started